

Task Manager App

A calendar-centric life dashboard: tasks, fitness, medication, payments, and expenses in one place



Prepared for Aswin • September 2026

1. Core concept

The app is a calendar-centric task manager. The **Home screen is a live calendar**, and four separate life-domain modules feed into it: **Fitness**, **Reminders** (which itself contains Pill, Payment, Appointment, and Custom), and **Expense**.

The calendar's role is strictly that of an **aggregator and preview layer** — it displays a compact signal of what's happening on a given day (an icon, a badge, a total) and lets the user drill into the owning module for full detail. It does not own or duplicate the underlying data itself.

Why this matters: keeping the calendar "dumb" (display-only) and the modules "smart" (own their data and logic) avoids the classic trap of the same information living in two places with two different edit paths — which is confusing and error-prone for users.

2. Navigation structure

Bottom navigation bar, four tabs:

Tab	Role
Home	Calendar view. Aggregates and previews entries from all other modules. No data is edited here except through contextual, date-based add flows.
Fitness	Workout logs and goals.
Reminders	Pill, Payment, Appointment, and Custom reminders — grouped under one tab, each entry type keeping its own icon for quick visual recognition.
Expense	Categorized spending ledger; rolls up into daily, weekly, and monthly totals.

Why Pill and Payment share a tab instead of getting their own

Structurally, a pill reminder and a payment reminder are the same object type: a due date/time with a "mark done" action. They differ by *domain* (health vs. money), not by *structure*. Splitting them into two separate bottom-nav slots would use up scarce navigation real estate for what is essentially the same interaction pattern twice. Grouping them under one "Reminders" tab — with each entry still carrying its own icon — keeps the nav minimal while preserving instant visual recognition of what kind of reminder it is.

3. The "+" add flow

Two entry points for adding something, both following a **module-first** principle — never a single flat list mixing all categories from every module together.

Global "+"

Tapping the general add button shows the module picker first (Fitness / Reminders / Expense), and only after a module is chosen does that module's own categories appear.

Date-specific "+"

Tapping a specific date on the Home calendar shows the module options directly — Pill, Fitness, Payment, Appointment, Custom — with that date already pre-filled in the new entry. This is the faster, more calendar-native path and is expected to be the primary way most entries get created.

4. Reminders module

Four sub-categories, each with a distinct icon so entries stay recognizable at a glance:

4.1 Pill MVP: manual entry v2: auto-lookup

Functions as a lightweight **medical ID** — designed with elderly users specifically in mind, so someone can open the app and show a doctor or pharmacist exactly what they're taking without needing a written prescription on hand.

Fields: medicine name, dosage, time(s) per day, expiry date, photo of the strip/box.

Phased build recommended. Auto-populating full drug details (company, strength, interactions) from just a typed name or a barcode scan requires integrating an external medicine/drug database — a real subsystem, not a form field. **Phase 1 (MVP)** ships with manual entry only, which already solves the core "show my doctor what I take" use case. **Phase 2** adds autocomplete/lookup once there's a real user base to justify the integration work.

4.2 Payment / Bill

Fields: payee, amount, due date, recurring flag (rent, EMI, subscription, utility bill, etc.).

Connects directly into the Expense module — see Section 6.3 for the linking behavior.

4.3 Appointment

Fields: title, location, time. (Doctor visits, meetings, and similar scheduled events.)

4.4 Custom

The catch-all category — this matters more than it might first appear, since no fixed category list will ever anticipate every use case a user has. Without an escape hatch here, users get frustrated trying to force-fit odd reminders into rigid categories.

The user picks any **emoji from their device's keyboard** to represent the entry, letting them build their own personal visual language over time (e.g. 🎸 = guitar practice, 📖 = study session) without the system needing to guess their taxonomy in advance.

4.5 Reminder timing — module-specific, not universal

A single generic "remind me at X time" setting doesn't fit every module well — Pills need multiple daily slots, Payments need advance warning rather than a single time, and Fitness/Custom just need a simple time. So timing configuration is tailored per module:

Module	Timing control
Pill	Times-per-day picker (supports multiple daily doses)
Payment	Days-before-due picker (advance warning, not a fixed time)
Fitness / Custom	Simple time picker

5. Custom category — grouping and tracking rules

5.1 Same-day, same-emoji grouping

If a user adds a new Custom task on a date where that same emoji is already used, it does **not** create a second top-level calendar entry. Instead, the new task is grouped as another item under that emoji's "bucket" for that day. Each grouped task within the bucket must carry a distinct timestamp, so multiple same-category tasks in one day stay individually identifiable.

This is effectively a lightweight **faceted classification system**: the emoji acts as the category facet, and the timestamp acts as the disambiguator within that category — the same underlying pattern used by well-designed tagging systems generally.

5.2 Calendar display for grouped entries

Rather than repeating the same icon multiple times on one date (which becomes visually noisy fast), the calendar cell shows the emoji **once with a count badge** — e.g. "👤 x3" — and tapping it expands the full list with individual timestamps.

5.3 Custom tracking

Custom entries are not just record-and-remind — they also support progress tracking, matching the depth given to the other modules. Since the system can't predict what an arbitrary user-invented category should measure, the tracking type is chosen **per item, at creation**:

- **Streak / checklist** — a simple did-I-do-it-today check.
- **Numeric** — a quantity, e.g. how long or how much.

6. Expense module

6.1 Categories

Food, Transport, Bills/Utilities, Shopping, Health, and an Other catch-all — mirroring the same "always keep an escape hatch" principle used in Reminders.

6.2 Calendar integration

- Tapping a specific date on Home shows that day's total spend, auto-populated from ledger entries — no need to open the full ledger just to see a daily figure.
- The calendar's month view shows a small summary column at the end of the month with the total spend for that month, helping users plan ahead for the next one.

6.3 Payment ↔ Expense linking

The problem this solves: without a link, a user could mark a bill "paid" in Reminders *and* separately log that same amount in the Expense ledger — double-counting it in the monthly total, which breaks the entire "control your spending" value proposition.

The rule: when a Payment reminder is marked paid, it automatically creates a linked Expense entry in the Bills category for that date and amount — the user never manually re-enters it. The Payment reminder is the source of truth for that expense line.

An opt-out toggle is available at the time a payment is created — "*count this toward my expenses?*" — for cases where a bill isn't the user's own personal spending (e.g. paying on behalf of a roommate, or a business-related expense).

Both paid and unpaid payment amounts appear in the monthly spend total — unpaid entries represent *committed*, upcoming spend rather than money already gone, which is more useful for forward planning than only showing actual spend. It's recommended that paid and unpaid amounts be visually distinguished in the totals view (e.g. solid color for paid, outlined/lighter style for unpaid) so users can tell the two apart at a glance.

7. Fitness module — open design area

This is the least-finalized module and needs a dedicated design pass. Two object types have been identified so far:

- **Workout log** — retrospective: "I did this today."
- **Goal** — prospective: "I'm aiming for this."

These represent two different mental models (looking back vs. looking forward), which suggests Fitness should get the same module-first sub-choice treatment as Reminders when adding an entry — "Log a workout" vs. "Set/check a goal" — rather than one flat screen mixing both.

Open questions still to resolve:

- Do goals drive what shows on the calendar (planned workout days), or does the calendar only reflect workouts that were actually logged after the fact?
- Is manual entry the permanent assumption, or is eventual sync with phone step-counters / wearables in scope? (This is the same type of build-effort decision as the Pill auto-lookup question in Section 4.1 — a real integration project, not a form field.)

8. Onboarding

New users are guided through setup rather than dropped onto a blank calendar with no direction — an empty Home screen with no prompt is a well-known drop-off trap, and this app in particular has real setup cost in its richer modules (especially Pill, with its dosage/expiry fields).

- Users are **forced through setup of at least 2 modules** before a skip option becomes available, so they leave onboarding having actually used the system rather than just been told about it.
- **If the user's age is above 45**, onboarding forces setup of **all 4 modules** instead of just 2 — extra hand-holding aimed at the group most likely to benefit from the app's higher-value features, particularly the Pill medical-ID function.

9. Deferred / v2 scope

Explicitly out of the MVP build, to be revisited once there's a real user base:

- Drug database auto-lookup for Pill entries (typed name or barcode scan → auto-filled medicine details).
- Wearable / step-counter sync for the Fitness module.

10. Still to design

- Fitness module detail — resolve the two open questions in Section 7.
- Home calendar screen visual layout.
- Full onboarding flow content — exactly what the 2 forced modules walk a new user through.